

Lab 7 — Multilayer Perceptron with Backpropagation (XOR) from Scratch

Aim

To implement backpropagation from scratch to train a Multilayer Perceptron (MLP) to learn the XOR function.

Algorithm / Procedure

1. Initialize the input, hidden-layer, and output-layer weights randomly.
2. Define the sigmoid activation function and its derivative.
3. Perform forward propagation:
 - Calculate hidden-layer output.
 - Calculate output-layer prediction.
4. Calculate the output error.
5. Backpropagate the error to the hidden layer.
6. Calculate weight updates using gradient descent with momentum.
7. Repeat the process for several epochs.
8. Predict the XOR outputs after training.
9. Display the predicted values.

Program

```
import numpy as np
# Sigmoid activation function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
# Derivative of sigmoid
def sigmoid_deriv(x):
    return x * (1 - x)
# XOR input and output
X = np.array([
    [0, 0],
    [0, 1],
```

```

    [1, 0],
    [1, 1]
])
y = np.array([
    [0],
    [1],
    [1],
    [0]
])
# Set random seed
np.random.seed(1)
# Initialize weights
w1 = np.random.uniform(-1, 1, (2, 4))
w2 = np.random.uniform(-1, 1, (4, 1))
# Momentum values
dw1_prev = np.zeros_like(w1)
dw2_prev = np.zeros_like(w2)
learning_rate = 0.5
momentum = 0.9
# Training
for epoch in range(10000):
    # Forward propagation
    h = sigmoid(np.dot(X, w1))
    o = sigmoid(np.dot(h, w2))
    # Output error
    err_o = (y - o) * sigmoid_deriv(o)
    # Hidden layer error
    err_h = np.dot(err_o, w2.T) * sigmoid_deriv(h)
    # Weight updates
    dw2 = learning_rate * np.dot(h.T, err_o) + momentum * dw2_prev
    dw1 = learning_rate * np.dot(X.T, err_h) + momentum * dw1_prev
    w2 += dw2
    w1 += dw1
    # Store previous updates
    dw2_prev = dw2
    dw1_prev = dw1

```

```
# Display final predictions
print("Predicted output after training:")
print(np.round(o, 3))
```

OUTPUT

```
... Predicted output after training:
 [[0.009]
  [0.992]
  [0.992]
  [0.008]]
```

Result

The Multilayer Perceptron was successfully implemented from scratch using backpropagation with momentum. The trained network successfully learned the non-linearly separable XOR function.